2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM) | 978-1-6654-9609-4/22/$31.00 ©2022 IEEE | DOI: 10.1109/SCAM55253.2022.00037

Flaky Test Sanitisation via On-the-Fly Assumption Inference for Tests with Network Dependencies

Jens Dietrich*, Shawn Rasheed†, Amjed Tahir†
*Victoria University of Wellington, Wellington, New Zealand
†Massey University, Palmerston North, New Zealand
jens.dietrich@vuw.ac.nz; s.rasheed@massey.ac.nz; a.tahir@massey.ac.nz

***Abstract*—Flaky tests cause significant problems as they can interrupt automated build processes that rely on all tests succeeding and undermine the trustworthiness of tests. Numerous causes of test flakiness have been identified, and program analyses exist to detect such tests. Typically, these methods produce advice to developers on how to refactor tests in order to make test outcomes deterministic.**

We argue that one source of flakiness is the lack of assumptions that precisely describe under which circumstances a test is meaningful. We devise a sanitisation technique that can isolate flaky tests quickly by inferring such assumptions on-the-fly, allowing automated builds to proceed as flaky tests are ignored. We demonstrate this approach for Java and Groovy programs by implementing it as extensions for three popular testing frameworks (JUnit4, JUnit5 and Spock) that can transparently inject the inferred assumptions. If JUnit5 is used, those extensions can be deployed without refactoring project source code.

We demonstrate and evaluate the utility of our approach using a set of six popular real-world programs, addressing known test flakiness issues in these programs caused by dependencies of tests on network availability. We find that our method effectively sanitises failures induced by network connectivity problems with high precision and recall.

***Index Terms*—flaky tests, non-determinism, regression testing**

I. Introduction

Flaky tests, i.e., tests with non-deterministic outcomes, are a widely known phenomena that causes significant problems in industry [1], [2], [3], [4]. Modern software development processes heavily depend on a high level of automation, and for many projects, automated regression testing remains the main means to demonstrate fitness for purpose. Therefore, it is desirable and often necessary for automated continuous integration (CI) that all tests succeed. With flakiness, this process becomes brittle – even if all tests succeed, it is unclear whether this is something that can be trusted, or whether tests have only accidentally succeeded.

Dedicating resources to fix flakiness is challenging as it is not clear to decision makers whether flakiness is caused by a fault in the program under test, the test itself, or the test environment or configuration that is being used. Fowler therefore suggested a two-staged process to deal with flaky tests [21]:

> *"Place any non-deterministic test in a quarantined area. (But fix quarantined tests quickly.)"*

This reflects the situation developers face when they have to deal with flaky tests. For instance, Raine reports about the practices followed at GitHub [3]:

> *"When we set out to build this new system, our intent wasn't to fix every flaky test or to stop developers from introducing new flaky tests. Such goals, if not impossible, seemed impractical. ... Rather, we set out to manage the inevitability of flaky tests."*

There are two strategies to deal with flakiness. Firstly, a practical approach to deal with flakiness as it occurs in a build is to trigger appropriate processes [22], [23]. Failing tests are observed and then re-evaluated, and if they eventually pass, they are then marked as *"success, but flaky"*. Projects can then respond to this by tagging those tests, quarantining them and/or automatically opening issues. The focus here is on the current build cycle. The second approach aims at finding and addressing the root causes of flakiness to deal with it *a priori*. Some of these being test order dependencies, concurrency, randomness, sensitivity to the test environment configuration, and dependency on network resources [5]. Detection usually utilises program analysis to inform refactoring [24], [8], [25].

We propose a novel approach to quarantine flaky tests in Java programs on-the-fly to make builds succeed without the need to rerun or rebuild. Our approach is based on a dynamic analysis inferring the assumption under which a failed test might pass, then disabling the test while also identifying the possible reason for flakiness. Analysis results are communicated in a way which allows processes or builds to proceed without requiring refactoring. This retains the advantage that cause detection has in terms of provenance, i.e., our approach is not black box, it produces information as to why a test was skipped to assist engineers to address the sources of flakiness in the long run by informing the refactoring of code, tests and configuration scripts. There are multiple use cases for this general approach for sanitising flaky tests due to root causes such as network dependency, test order dependence and environment dependency. For instance, tests could be disabled if (instrumented) test runs detect certain configuration (JVM versions, hardware, OS etc) either directly known or inferred to lead to flaky behaviour, or if executions of tests interfering with state a test relies on have been recorded.

We specifically illustrate the utility of our approach to deal with network dependencies in tests. This has been identified as one of the major sources of test flakiness [26], [27], [5] and Luo et al. [26] classify 6% of the flaky tests fixes they studied to be due to network dependencies. Our approach is particularly suitable here, as it is difficult or even impossible

2470-6892/22/$31.00 ©2022 IEEE
DOI 10.1109/SCAM55253.2022.00037

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 12,2026 at 15:39:53 UTC from IEEE Xplore. Restrictions apply.

for engineers to write test fixtures or assumptions to reliably control or check the availability of networked resources when tests are run.

This paper is organised as follows. We discuss the background in Section II and develop the conceptual foundations for our approach in Section III. We then discuss the methods and tools we have developed in Section IV, followed by a discussion of how to apply them to deal with network dependencies in Section V. Section VI contains the evaluation of our approach on a set of real-world programs with known flakiness issues. This is followed by a discussion of related work in Section VII and a brief conclusion.

We have included the scripts, datasets and results referenced in this paper in a replication package: https://bitbucket.org/unshorn/flaky-test-sanitisation/.

II. Background

A. Assumptions

A unit test (written in *JUnit*) has a simple structure containing some computation, and some assertions to check the results of this computation against some expectations (oracles). A simple test is shown in Listing 1.

```
1 @Test public void test() {
2    int result = calculator.divide(4,2);
3    assertEquals(2,result);
4 }
```

Listing 1: A simple *JUnit* test

The lifecycle of classes / objects to be tested (such as `calculator`) is usually managed outside the actual tests in a test *fixture*. For instance, in *JUnit*, those are methods annotated with `@BeforeEach`, `@BeforeAll`, `@AfterEach` and `@AfterAll`, respectively [1]. A *test run* is a function mapping a set of tests to a *success* or *failure* state, i.e., $\rho : T \rightarrow \{success, failure\}$. Flakiness occurs if there are two runs ρ_1 and ρ_2 producing different results for at least one test $t \in T$, i.e., $\rho_1(t) \neq \rho_2(t)$ [2].

Many testing frameworks, including *JUnit*, define an additional state to describe the outcome of executing a test – *error*. For instance, if the `divide` function used in Listing 1 was invoked with a second parameter 0 in a test, then the test might result in an exception or error. Unless a test is specifically setup to test for this, *JUnit* would flag this outcome as *error*, handling it slightly differently from *failure*. This raises the question whether a test suite should still be considered as flaky if between any two runs, the same tests pass (*success*), but the execution of the tests sometimes results in *failure*, and sometimes in *error*. Failure and error are similar in that they reflect an unexpected state reached by the execution of test. The difference is merely whether this state is made *explicit* (leading to *failure*), or *implicit* (leading to *error*). The latter means that there is an implicit oracle [28] that the execution of the functionality tested *should not* result in an unhandled exception or error.

There is a fourth state that results from violation of *assumptions*. Consider the test shown in Listing 2, and assume that the calculator to be tested is backed by a web service. Then the test is only meaningful if this service is reachable, and an assumption is used to express this.

```
1 @Test public void test() {
2    assumeTrue(networkIsAvailable());
3    int result = calculator.divide(4,2);
4    assertEquals(2,result);
5 }
```

Listing 2: A JUnit test with assumptions

Assumptions are not a widely utilised feature. Table I demonstrates this – it shows the number of test classes [3], used standard assertions (*assrt.* column) and assumptions (*assum.* column) [4] used and ignored or disabled tests (in the *ign./dis.* column, these can be considered as the strongest assumptions that always fail), for both *JUnit4* and *JUnit5* APIs, in the programs used in the evaluation in Section VI. Those features were extracted using an ASM-based bytecode analysis [29] on the compiled test classes.

TABLE I: Use of testing features in programs

program	classes	tests	assrt.	assum.	ign./dis.	cond.
biojava	283	1,325	4,473	7	30	0
jabref	457	3,442	5,422	4	20	0
jmeter	222	1,584	3,290	5	5	4
jsoup	47	928	2,848	0	4	0
pdfbox	205	930	3,202	4	2	0
swagger-parser	1	25	0	0	0	0

```
1 @Test void isNonHeadlessPDFBoxTest() {
2   final String[] args = {"debug"};
3   assumeFalse(GraphicsEnvironment.isHeadless(), ..);
4   assertDoesNotThrow(() -> {PDFBox.main(args);});
5   ..
6 }
```

Listing 3: *pdfbox* test with assumption

An interesting use of assumptions aligned with the approach presented here can be found in the tools module of *pdfbox* (class `..PDFBoxNonHeadlessTest`), shown in Listing 3. This test is only meaningful if it is executed in a non-headless mode as a graphical user interface is created. This is often not the case when the test is evaluated within a container as part of a CI process. Nevertheless, the test is still useful when executed on a developer desktop.

JUnit5 introduced conditional tests using additional annotations to facilitate writing assumptions using

[1]Throughout the paper, when we refer to *JUnit* we mean *JUnit5* unless explicitly mentioned otherwise. Most concepts discussed have counterparts in *JUnit4* and older versions and are also present in alternative frameworks for automated unit testing.

[2]This nondeterminism could also be modelled by using a second *environment* parameter, as done in [6]. We chose to consider two runs as different functions for the sake of a compact presentation.

[3]This refers to the number of `.class` files found containing methods annotated as tests, methods with `@Test` or `@ParameterizedTest` annotations

[4]Standard here refers to the assumptions and assertions defined as static methods in `org.junit.jupiter.api.Assumptions` (*JUnit5*), `org.junit.Assume` (*JUnit4*), `org.junit.jupiter.api.Assertions` (*JUnit5*) and `org.junit.Assert` (*JUnit4*) respectively, we are counting the call sites for those methods in the respective classes.

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 12,2026 at 15:39:53 UTC from IEEE Xplore. Restrictions apply.

common use cases, such as dependencies on the OS (`@DisabledOnOs`, `@EnabledOnOs`) or the JRE (`@DisabledOnJre`, `@EnabledOnJre`) used. *Spock* [5] has the `@Required` and `@IgnoreIf` annotations that take an expression (closure) as a parameter for a similar purpose. Occurrences of those features in the programs studied are listed in the *cond.* column in Table I.

The semantics of assumptions is defined in the *JUnit* documentation as follows:

> *A failed assumption does not mean the code is broken, but that the test provides no useful information. Assume basically means "don't run this test if these conditions don't apply". The default JUnit runner skips tests with failing assumptions.* [6].

That is, a dedicated *skip* state is used by test runners to mark tests failing assumptions [7].

The duality of assumptions and assertions stipulates that tests follow the general pattern of a specification, often represented by the triples in Hoare-Floyd logic [30], with assumptions corresponding to preconditions, and assertions to postconditions. This relationship is well-known and has been exploited, for instance, to generate tests from formal specifications [31], [32]. However, there are some important differences that are relevant for our discussion here: in Hoare-Floyd logic there is no dedicated state to express failed preconditions, whereas tests use a dedicated multivalued logic to deal with failed assumptions with a dedicated *skip* (*unknown*) state. What is more, tests cannot be assumed to be side-effect-free functions, they often modify memory or external resources. Finally, some tests are inherently concurrent even if they are not executed concurrently. An example for this is discussed later in Section V, where a test execution relies on network connectivity provided by an OS process which cannot be controlled by a fixture, or even on the availability of DNS, routers and servers. There is no counterpart for this in Hoare-Floyd logic, although there are extensions to model concurrency [33], [34].

B. Notions of Flakiness

Usually, flakiness is defined as non-deterministic test outcome where the same test sometimes succeeds and sometimes fails (which is signalled by either the *failure* or the *error* state). By also taking the *skip* state into account, we may also consider a weaker notion of flakiness. *Weak flakiness* occurs if the outcome of any test evaluated changes. In particular, this includes changes of test outcomes between runs from *success* to *skip*, and vice versa.

When considering the default configuration of many build systems, the notion of *weak flakiness* may not be very interesting, as it does not carry the risk of builds failing. For instance, consider the most widely used Java build tool: Apache Maven [8], with tests being executed using the test lifecycle phase with the default *surefire* configuration. While any test resulting in a *failure* or *error* state will lead to a build failure, tests resulting in a *skip* state do not, thus builds can proceed.

On the other hand, if weak flakiness is detected and a respective signal is emitted, processes can be customised to respond to this, for instance, by creating issues. This aligns with the idea of emitting orange signals (in addition to green / red) in order to break free of the ''*boolean straight jacket*'' assumption [35].

C. Explicit vs Implicit Assumptions

The impact of evaluating assumptions and assertions on state can be summarised in Table II.

TABLE II: Test result states in JUnit

condition	explicit	on success	on failure
assumptions	yes	continue	*skip*
assertions	yes	*success*	*failure*
	no	*success*	*error*

What is noticeable is that there is no provision for implicit assumptions. The (meta-)assumption is that there is no need for this, as all assumptions can be made explicit. This hinges on the expectation that an engineer can completely control and check the program and the environment it interacts with. We believe that this expectation is flawed for two reasons: (1) many parameters a test execution relies on cannot be controlled and not even be checked and (2) even if this was possible, this would require an unrealistic level of understanding of the program and its interactions with the environment by the engineer(s) in charge of testing.

However, these insights also provide the opportunity to explore the very notion of implicit assumptions further, and investigate whether this can lead to a novel approach to tackle certain kinds of flakiness.

III. Strengthening Assumptions as Deflaking Strategy

A. Deflaking Strategies

Given the structure of a test, there are multiple ways to tackle flakiness. This is very much a "horses for courses" scenario, i.e., different approaches might be suitable for different situations. An empirical study by Luo et al. [26] has shown several methods are used by developers to address flakiness. Possible strategies include:

- **Modifying the Program under Test** in order to improve testability [36], examples include avoiding the unnecessary use of concurrency and randomness, and controlling aliasing in order to reduce dependencies between tests.

[5]https://spockframework.org/

[6]https://junit.org/junit4/javadoc/4.12/org/junit/Assume.html

[7]There are other reasonable interpretations of failed assumptions, for instance, a test could be interpreted using the semantics of the material implication, meaning that a test would succeed if its assumptions failed. The introduction of another state however has the advantage of adding some provenance to testing.

[8]https://www.jetbrains.com/lp/devecosystem-2021/java/#Java_which-build-systems-do-you-regularly-use-if-any

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 12,2026 at 15:39:53 UTC from IEEE Xplore. Restrictions apply.

- **Modifying the Test Fixture**, examples include cleaning up resources (heap or external) used by tests that may lead to test dependencies, or setting random seeds.
- **Reconfiguring the Test Environment**, examples include using fork options in tools like Maven surefire, and using a customised JVM to better control test dependencies [37]).
- **Weakening Assertions**, examples include the introduction of deltas when comparing numerical values, and replacing fixed oracles with statistical oracles [38], [39].

B. Strengthening Assumptions

There is another option to deal with flakiness: the strengthening of assumptions. Given that we can interpret no assumption as the weakest possible assumption that always evaluates to true, this usually means adding assumptions to tests.

Strengthening assumptions is particularly suitable to state dependencies on relevant aspects of the test environment that cannot be controlled in fixtures.

C. Static vs Dynamic vs External Assumptions

Looking at assumptions, we can broadly classify them as follows: **Static assumptions** that always evaluate to *true* or *false* – this includes the empty assumption always evaluating to true and the strongest possible assumption defined by the special annotations like `@Ignore` or `@Disable` that can never be satisfied. On the other hand, **dynamic assumptions** are evaluated before the actual test is executed, but the result of the evaluation may be different for each execution. This is the kind of assumption supported by the assumption APIs (`Assume` and `Assumptions`, respectively) in *JUnit*. Finally, there are **external assumptions** – assumptions related to the execution environment of the test that cannot be checked using a unit testing API. However, those assumptions may still emit observable effects and therefore actionable signals during test execution (i.e., they require a *speculative execution* of the test to be observed). The main use case is the presence of reliable network connections during testing that cannot be checked or controlled via some testing API, but has a direct impact on the outcome of tests. This will be discussed in detail in Section V.

The rest of this paper focuses on external assumptions. Support for external assumptions is provided by means of *sanitisers*, testing framework extensions that intercept test processing, and can check inferred assumptions on-the-fly, i.e., without an explicit representation of these assumptions in tests. This means that flaky tests that would otherwise result in *failure* or *error* are skipped, allowing builds to proceed (but keeping a record of those tests).

IV. Sanitisers

A. Sanitisers as JUnit5 Extensions

We have developed a proof-of-concept implementation to sanitise [9] flaky tests called *saflate*. *Saflate* sanitisers intercept the execution of tests, inspect the state of tests, checks inferred assumptions and depending on those conditions will change the state of some tests from *failure* or *error* to *skip*. In *JUnit5*, states are set and propagated via special errors: `java.lang.AssertionError` are used to signal failed tests, whereas `org.opentest4j.IncompleteExecutionException` and its subclasses are used to signal skipped and aborted [10] tests. Changing the test state is therefore a matter of intercepting test processing, catching certain exceptions or errors, and rethrowing others.

Saflate is implemented using the *JUnit5* extension model [40, Sect 5], it requires *JUnit 5.6.0* or better. Extensions implement some test framework interfaces, and be deployed programmatically, declaratively using annotations, or automatically as services. The annotation-based approach is class-based, for instance, the code in Listing 4 will install the network dependency annotation extension discussed in Section V for the tests within the class `MyTests`.

```
1 import .. .SanitiseNetworkDependenciesExtension;
2 import org.junit.jupiter.api.extension.ExtendWith;
3
4 @ExtendWith(SanitiseNetworkDependenciesExtension.class)
5 public class MyTests {
6    @Test public void test() {
7       ..
8    }
9 }
```

Listing 4: Using the network dependency sanitiser extension

Using the automated service-based mechanism, developers have to add *saflate* as a dependency to the project, without the need for code refactoring. *JUnit5* will then automatically discover the service and inject it into the test processing pipeline. The only additional action required is to set the `junit.jupiter.extensions.autodetection.enabled` system property to `true`, this can be done via a JVM argument [11].

Some sanitisers need to instrument existing classes. For the sanitisers discussed here, this is done dynamically using *ByteBuddy* [12] without the need of installing an agent. The instrumentation is triggered by the static blocks of extension classes.

The semantics of the sanitiser is defined by the following two rules:

R1 If the test state is *failure* or *error*, and an inferred assumption is not satisfied, then its state is set to *skip*, and the test proceeds.

R2 Otherwise, the test proceeds uninterrupted.

When the state is changed to *skip*, an instance of the respective exception that signals the skip state within the respective testing framework is created and thrown. This facilitates *provenance* – sanitisers add a descriptive message as to why the test was skipped that is then included in

[9] We use sanitise in the sense of "to make (something, such as text) more acceptable by removing, hiding, or minimizing any unpleasant, undesirable, or unfavourable parts' [https://www.merriam-webster.com/dictionary/sanitize]

[10] Aborted means a failed assumption, whereas skipped means ignored due to an annotation. We refer to both situations as skipped throughout this paper.

[11] For instance, in Maven, this can be configured in the *surefire* plugin used for testing

[12] https://bytebuddy.net/

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 12,2026 at 15:39:53 UTC from IEEE Xplore. Restrictions apply.

generated reports and communicated to users, where it can inform refactoring.

Multiple such extensions can be combined by chaining the respective extensions. If multiple preconditions associated with extensions are violated, only the first one encountered will change the state of the test to *skip*, as tests with a state *success* and *skip* are not intercepted. This has some impact on provenance, which is provided through the messages of the exceptions signalling that an assumption was not satisfied: only the details about the first violated assumption are signalled. When extensions are deployed programmatically or using annotations, the user has control over the order in which extensions are being processed [40, Sect 5].

B. Dealing with Test Fixtures

The extensions consider the `@BeforeEach` and `@AfterEach` fixtures as part of the test. This is consistent with standard *JUnit5* behaviour – if either of these methods results in an *error*, so will the respective tests. However, `@BeforeAll` and `@AfterAll` are not intercepted – errors occurring in those methods lead to all tests in scope being skipped.

C. Backporting Extensions as JUnit4 Rules

Some programs we used to evaluate our approach employ the older *JUnit4*. For this purpose, we backported saflate extensions as *JUnit4* rules. In order to deploy those rules, tests have to be modified by inserting those rules as fields since *JUnit4* does not support the service-based auto-deployment available for *JUnit5* extensions.

D. Implementing Sanitisers as Spock Extensions

Spock [41] is an increasingly popular testing framework for Java and Groovy, written in those languages. It supports an annotation-based extension mechanism. The semantics of an annotation extending *spock* can be provided by an *interceptor* that intercepts the testing pipeline and can change the state of a test. We followed the overall designing of built-in `@PendingFeature` annotation discussed earlier and again in Section VII-D.

V. Application: Sanitising Network Dependencies

A. Motivation

Modern applications are increasingly networked as remote services and resources are both provided and consumed. This causes challenges for testing as tests rely on a stable network connection to succeed. Problems related to network connectivity have recently been identified as a major cause for failing and restarted builds on Travis CI [17], and in organisations like Uber [4] and Mozilla [42].

The provision of stable network connections can generally not be ensured in a test fixture. There is no reliable way for engineers to write assumptions that certain tests are only executed when the network is available [13]. And even if there was such an API, it would be of limited use, as network connectivity could be lost when a test is executed, but after the respective assumption has been checked.

Many programmers prefer to mock APIs and services relying on network connectivity using approaches such as dynamic mock libraries (*mockito*[14] etc), static network-specific libraries such as Springs servlet API mock objects [15], or more complex frameworks based on service virtualisation [43]. However, tests directly relying on resources accessed via the network are still common. This makes network dependencies a major source of flakiness [26], [27], [5].

B. An Optimistic Approach: Hoping for the Best

Consider the test[16] shown in Listing 5, taken from the popular *jabref* project - an application to manage bibtex databases.

```
1  package org.jabref.logic.help;
2  ..
3  class HelpFileTest {
4    private final String jabrefHelp = "https://docs.jabref.
       org/";
5    static Stream<HelpFile> getAllHelpFiles() {
6      return Arrays.stream(HelpFile.values());
7    }
8    @ParameterizedTest
9    void referToValidPage(HelpFile help) throws IOException {
10     URL url = new URL(jabrefHelp + help.getPageName());
11     HttpURLConnection http =
12       (HttpURLConnection) url.openConnection();
13     http.setRequestProperty("User-Agent",URLDownload.
        USER_AGENT);
14     assertEquals(200, http.getResponseCode(), ..);
15   }
16 }
```

Listing 5: A *jabref* test with a network dependency

The help system uses online resources, and this functionality is tested here. From the project's point of view, this has merits: an online help system is easier to maintain, and this avoids adding (potentially large binary) resources to the repository and to the distribution. But if those tests fail due to the network not being available then this does not indicate that the code is incorrect, it merely states that these particular tests cannot be evaluated at the current time, and therefore should be skipped. Overall, the approach taken here is *optimistic*, in the sense that the developers expect the network to work and the site to be available and reachable.

C. A Pessimistic Approach: Reducing Coverage (but "not reliant on the vagaries of the internet")

Now consider a following test[17] (Listing 6) in *jsoup* - a popular HTML parser library. Here, developers took a *pessimistic* approach by disabling network-sensitive tests. While this reduces flakiness caused by network problems, it also unnecessarily removes valuable tests when the network is available as the assumption is static.

[13]This topic frequently comes up in discussions, for instance see https://stackoverflow.com/questions/28397007/how-to-simulate-lack-of-network-connectivity-in-unit-testing.

[14]https://site.mockito.org/

[15]https://docs.spring.io/spring-framework/docs/current/reference/html/testing.html#mock-objects

[16]https://github.com/JabRef/jabref/blob/bb011c9313367a28990ae213b3920fe6cd10d1dc/src/test/java/org/jabref/logic/help/HelpFileTest.java

[17]https://github.com/jhy/jsoup/blob/master/src/test/java/org/jsoup/integration/UrlConnectTest.java

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 12,2026 at 15:39:53 UTC from IEEE Xplore. Restrictions apply.

```
1  package org.jsoup.integration;
2  ..
3  /**Tests the URL connection. Not enabled by default, so
4   tests don't require network connection. .. */
5  @Disabled // ignored by default so tests don't require
       network access. comment out to enable. todo: rebuild
       these into a local Jetty test server, so not reliant on
        the vagaries of the internet.
6  public class UrlConnectTest {
7    ..
8    @Test public void fetchBaidu() throws IOException {
9      Connection.Response res = Jsoup
10       .connect("http://www.baidu.com/")
11       .timeout(10*1000).execute();
12     Document doc = res.parse();
13     ..
```

Listing 6: *jsoup* test with network dependency

The two examples illustrate that developers often do not have good choices – either test less, or accept flakiness which leads to failing tests. This is the problem we are trying to address in this work.

D. Hidden Network Dependencies

Network dependencies can be subtle, and the engineer writing tests may not always be aware of their presence. For instance, consider the case of XML parsing. Often, XML documents reference some remote schema (XSD, DTD or similar) using a URL. Many applications use validating parsers to check that documents comply to a vocabulary. This requires network access at the time of validation. Even if the parser validation is disabled, schema access may still be required to resolve entity references. Other subtle causes of network-related flakiness are (tight) connection timeouts and port collisions.

E. Implementation

The condition enforced by the sanitiser is that no exception indicating a network problem must have occurred during the execution of the test. We consider three such exceptions (including subclasses of those listed) related to network availability:

1) `java.net.SocketException`
2) `java.net.UnknownHostException`
3) `java.net.NoRouteToHostException`

We implemented a sanitiser (called *saflate*) for network dependencies as both a *JUnit5* extension and a *JUnit4* rule, and an experimental port as a *Spock* extension. The list of exceptions to be sanitised is provided by a service, allowing the list to be extended by third parties. The standard Java `ServiceLoader` facility is used for this purpose, i.e. extension can be provided as libraries that advertise the implementation of the `NetworkExceptionProvider` interface in the component manifest. *saflate* source code is available on a public GitHub repository [18], and the deployed binaries are available in the Maven repository [19]

The sanitisers instrument the constructors of the respective exception classes. When the creation of such an exception is encountered, a flag is set to communicate to the test that a network exception has occurred. When a test fails or results in an error, then the test state is changed to *skip* if either (1) the test resulted directly in a network exception visible in the stack trace and the test is in an *error* state or (2) a network exception creation was recorded during test execution, and the test is in a *failure* state.

Communicating the state from the network exception instantiation to tests has to deal with classloader issues as those exceptions are loaded by the system classloader. We solved this problem by using a system property with a dedicated name in order to communicate properties between those exceptions, and the customised test execution. The property can also incorporate the thread ID in case tests are executed using thread-level concurrency [20]. This is an experimental feature introduced in *JUnit5*, but is used by both *pdfbox* and *jabref* in order to improve test performance.

Note that the condition we enforce is not a precondition as we can only detect a violation after the test has been at least partially executed, i.e., *a posteriori*. We can consider the execution of the test as speculative execution if a precondition holds, but interrupting a test if we find out later that this was not the case, relying on a fixture to rollback, i.e., to reset the state of the test. A clean precondition check would be certainly preferable, however, there is no way to check this, and then also to ensure network connectivity as a guarantee during the execution of the test.

F. Limitations

This approach has the following limitations that may result in false positives (unnecessarily sanitised tests) and false negatives (tests that should be sanitised, but are not).

Saflate will suffer from false positives, if network exceptions are created but not actually being thrown or intentionally thrown to test the exceptions (e.g. using `assertThrows`), but the test nevertheless results in *error* or *failure* for some other reason. We consider this scenario as very unlikely and suggest mitigating this by using the more fine-grained annotation-based deployment if such tests are used.

On the other hand, *saflate* will suffer from false negatives if support for test concurrency is enabled and network exceptions occur in background threads. This may result in tests not being sanitised. A second potential source of false negatives are network exceptions thrown by native methods, such as `java.net.Inet6AddressImpl::lookupAllHostAdd`, as the respective instantiations are not instrumented. This would only result in a false negative if the respective test resulted in *failure* (not *error*). It is possible to mitigate this by using a more sophisticated instrumentation that injects probes at the call / allocation sites of those constructors. This however is likely to increase the performance overhead.

[18]https://github.com/jensdietrich/saflate

[19]https://search.maven.org/search?q=io.github.jensdietrich.saflate

[20]This behaviour is by default disabled, but can be enabled by setting the system property `saflate.supportconcurrent-test-execution` to `true`

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 12,2026 at 15:39:53 UTC from IEEE Xplore. Restrictions apply.

The evaluation reports recall and precision in order to assess the impact those limitations have in practice. While we find instance of both false negatives and false positives, their relative numbers remain very low as we will demonstrate in the next Section.

VI. Evaluation

A. Dataset

Finding open-source projects with tests that experienced flaky behaviour due to network issues is challenging, as we noticed that such flaky behaviour might not appear until an interception to the network connection occurs if these tests (or the code being tested) rely on a network connection.

For our experiment, we constructed a dataset of projects with network dependencies from GitHub. We aimed to select only projects that are popular (with at least 300 stars) and with a decent history (over 2 years of age). Our goal is to select projects that are currently active and frequently maintained, thus we can report any flakiness we find to the maintainers.

We first searched for projects with open issues that are related to tests with flaky network behaviour. We used a simple search string *"flaky + network"*, but this search, based on the results in the first 10 pages, did not retrieve any relevant results. We then searched for issues that included the specific network-related exceptions that we intercept: *"flaky + (UnknownHostException OR NoRouteToHostException OR SocketException)"*. We identified only one project with a current open issue with a network-related flaky behaviour (*biojava*) that also met our selection criteria. We then decided to expand our search by conducting a general search on GitHub to identify tests that use specific HTTP GET requests such as `java.net.URL::openconnection` and `java.net.URL::openStream`. Based on this, we identified five additional projects (*jabref*, *jsoup*, *pdfbox* and *swagger-parser* and *jmeter*). The six projects are listed in Table III [21].

B. Experimental Setup

We run each program with four different configurations with and without the network enabled, and with and without the *saflate* sanitiser. For *pdfbox* and *jabref*, we enabled *saflate's* support for parallel test execution, discussed in Section V-E. Experiments were run in Docker containers that are made available in our replication package [23]. The host used is a computer with a 3.2 GHz 6-Core Intel Core i7 CPU running Docker Engine - Community 20.10.5.

C. Results

Table IV and Table V summarises the results obtained from the evaluation of the three programs [24]. Table VI lists the precision and recall scores for sanitisation. The results reported are averages taken over 10 runs - numbers in brackets indicate that those are average numbers with some variation between runs. We assess precision and recall as follows. The *sanitised tests* t_s are the tests that result in failure or error in all runs when the network is off without sanitisation, but are always skipped when sanitisation is used and the network is off. *Relevant tests* t_r are the test that always succeed when the network is available and sanitisation is off, but always result in failure or error when the network is off and no sanitisation is used. Precision is then defined as $|t_r \cap t_s|/|t_s|$, and recall as $|t_r \cap t_s|/|t_r|$. Multiple runs are used to deal with other sources of flakiness that may be present. These metrics are based on the assumption that we can set up an experiment that can control access to networked resources by disabling the network in the (docker-based) setup – this may not work if the problem is not the connection, but the availability of the server. An indicator of this is when tests already fail when the network is available, and can be sanitised successfully.

The results indicate that generally *saflate* is effective in sanitising tests failing or resulting in error due to network availability problems and poses no significant overhead. For *jsoup*, *pdfbox*, *swagger-parser* and *jmeter*, the results are perfect in the sense that *saflate* sanitises exactly the tests that stop succeeding when the network is disabled. Interestingly, in case of *swagger-parser*, *biojava*, *jabref* and *jmeter*, disabling the network results in failures as opposed to errors. I.e., when intercepting test processing, the error stack trace containing references to the network connection may not be available. This justifies our choice to use a dual strategy – instrument the allocation sites of network exceptions and analyse stack traces when available.

We discuss selected results in more detail in the next section.

D. Discussion

1) biojava: In *biojava*, there are 17 failures and 281 errors when the network is down. Sanitisation successfully picks up all these tests except for eight (one failure and seven errors). Closer inspection reveals that six of these tests are in the class `TestMultipleAlignmentWriter` (package names omitted for brevity), and the network-related error occurs in the constructor for the class. Fixture setup in the constructor of test classes is not a good practice, and a better fix for this would be to refactor the test class that would also allow sanitisation to intercept it. The other false negative, which is a test error is in `TestProteinSuperposition`, which is caused by a network error that the sanitisation fails to intercept (in a method annotated with `@BeforeClass`). The test failure, which is also a false negative occurs in `FileDownloadUtilsTest.URLMethods::pingGoogleOK`. This illustrates the second source of false negative discussed in Section V-E – an `UnknownHostException` is thrown in a native method (`Inet6AddressImpl::lookupAllHostAddr`), and the test swallows the exception (stacktrace) (`FileDownloadUtils::ping` returns `false` when

[21] For *swagger-parser*, we only consider the *swagger-parser* module. There are three other modules in this project, all using *TestNG*, which does not offer suitable extension points to port *saflate*, so we had to exclude these modules. The *swagger-parser* module still uses *TestNG* assertions, however, they are fully compatible with *JUnit4*.

[23] https://bitbucket.org/joe-bloggs/flaky-test-sanitisation/

[24] Test counts appear to be inconsistent with Table I. This is caused by parameterised tests – the static analysis counts them as single tests, whereas test execution reports each instantiation separately.

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 12,2026 at 15:39:53 UTC from IEEE Xplore. Restrictions apply.

TABLE III: Programs with tests depending on network connectivity

program	description	tag	GitHub stars	testing framework	concurrent tests	repo url
biojava	a set of tools to process biological data	biojava-6.0.4	486	junit 5 and 4	no	https://github.com/biojava/biojava
jabref	citation and reference management tool	v5.5	2.5k	junit 5	yes	https://github.com/JabRef/jabref
jmeter	performance and load testing tool	rel/v5.4.3	6k	junit 5 and 4 spock[22]	no	https://github.com/apache/jmeter
jsoup	Java HTML parser	jsoup-1.14.2	9.4k	junit 4	no	https://github.com/jhy/jsoup
pdfbox	tool for working with PDF documents	2.0.24	9.4k	junit 5	yes	https://github.com/apache/pdfbox
swagger-parser	parser for OpenAPI definitions	v2.0.18	559	junit 4	no	https://github.com/swagger-api/swagger-parser

TABLE IV: Test results without sanitisation. Results are reported across 10 test runs, numbers in brackets means that we observed some variation across runs and report the mean. The number of weakly flaky (w. flaky) and flaky tests across those runs are reported as well. Runtimes are reported in seconds (rt(s)).

program	tests	unsanitised											
		network on						network off					
		failure	error	skipped	w. flaky	flaky	rt (s)	failed	error	skipped	w. flaky	flaky	rt (s)
biojava	1,469	1	0	30	0	0	(1102.3)	17	281	33	2	2	(117.2)
jabref	7,586	1	0	19	0	0	(173.2)	57	0	19	0	0	(108.3)
jmeter	3,545	1	0	13	0	0	(96.2)	4	0	13	0	0	(349.4)
jsoup	930	0	0	3	0	0	(3.0)	0	1	3	0	0	(2.8)
pdfbox	1,881	0	0	2	0	0	(82.2)	0	35	2	0	0	(67.5)
swagger-parser	25	0	0	0	0	0	(3.6)	2	0	0	0	0	(2.0)

TABLE V: Test results with sanitisation. Results are reported across 10 test runs, numbers in brackets means that we observed some variation across runs and report the mean. The number of weakly flaky (w. flaky) and flaky tests across those runs are reported as well. Runtimes are reported in seconds (rt(s)).

program	tests	sanitised											
		network on						network off					
		failure	error	skipped	w. flaky	flaky	rt (s)	failed	error	skipped	w. flaky	flaky	rt (s)
biojava	1,469	1	0	(30.1)	1	0	(1228.1)	1	7	323	2	0	(227.7)
jabref	7,586	0	0	20	0	0	(190.0)	0	0	76	0	0	(109.5)
jmeter	3,545	1	0	13	0	0	(122.9)	1	0	16	0	0	(355.5)
jsoup	930	0	0	3	0	0	(3.3)	0	0	4	0	0	(3.1)
pdfbox	1,881	0	0	(2.1)	1	0	(85.5)	0	0	37	0	0	(70.3)
swagger-parser	25	0	0	0	0	0	(4.3)	0	0	2	0	0	(2.7)

TABLE VI: Precision and recall for sanitisation.

program	precision	recall
biojava	0.996	0.976
jabref	0.982	1
jmeter	1	1
jsoup	1	1
pdfbox	1	1
swagger-parser	1	1

an exception is encountered). As a result, the exception cannot be sanitised. There is also a false positive, `PDBStatusTest::testGetCurrent`, which is the test that fails in the baseline configuration. This test accesses the network but the failure is due to unexpected data from a REST API.

2) jabref: For *jabref*, there are 56 relevant tests and also 1 failure in the baseline configuration. The failure in the baseline configuration is also sanitised, which we observe as the single false positive. The particular false positive is a single test, `RemoteSetupTest::testPortAlreadyInUse`, which fails and when *saflate* sanitisation is present, it is skipped. The failure is not network related and it is due to a bug in the test's use of *mockito*. As all relevant tests are skipped by the sanitisation, there are no false negatives, leading to perfect recall and a precision of 0.982 due to the false positive.

3) jmeter: There are 3 relevant tests in *jmeter*, all of which are failures in the Spock specification, `DNSCacheManagerSpec`. An additional failure is also present in the baseline configuration. Sanitisation successfully skips all 3 relevant cases. There are no false positives, resulting in perfect recall and precision for the program with sanitisation.

4) jsoup: In *jsoup* there are 3 skipped tests in the baseline and one test case, `ConnectTest::handlesUnknownEscapesAcrossBuffer`, that terminates with an error when the network is off. This test is skipped with sanitisation on.

5) pdfbox: There are 35 relevant tests (i.e., tests that should be sanitised) as these tests result in error when the network is off whilst they succeed with the network on. The sanitisation successfully detects these tests and skips them. The two additional skipped tests, which are already in the baseline run (network on, sanitisation off) are tests in `PDFBoxNonHeadlessTest` guarded by the assumption `assumeFalse(GraphicsEnvironment.isHeadless())` that fails when running the tests in a docker container. This is a good use of assumptions.

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 12,2026 at 15:39:53 UTC from IEEE Xplore. Restrictions apply.

6) swagger-parser: For *swagger-parser*, the two cases which fail when the network is off are successfully skipped with *saflate* on, there are no false positives resulting in perfect precision and recall.

E. Performance

The overhead of using *saflate* is generally low. Performance data (in terms of runtime) is included in tables IV and V. The values reported are averages across runs. There are significant differences in the overhead that *saflate* adds and the overhead added depending on whether the network is on or off. This depends on multiple factors such as the speed of network access when the network is available, timeout settings for connections and reconnect set up when the network is unavailable. We observed average overheads of 14% when the network is on and 25% when the network is off. For three programs, *jabref*, *jmeter* and *pdfbox*, the overhead is less than 10% for both configurations.

We deem this as acceptable, and further optimisations are possible, such as managing timeouts and reconnect settings in the application, or using targetted annotations instead of the global deployment of *saflate* for *JUnit5*.

VII. Related Work

A. Flaky Test Detection Techniques

There are a number of techniques that have been proposed to detect flaky tests (mostly by looking at the change of the test outcome), with some of these tools able to identify the cause of flakiness. Often, tools focus on specific causes of flakiness, such as concurrency or order-dependency.

DeFlaker [44] attempts to avoid expensive reruns by detecting if test failures are due to flaky test behaviour without rerunning the entire test suite. DeFlaker works by recording the coverage of latest code changes and flags any newly failing test that did not exercise changed code as flaky. FlakyLoc [12] does not detect flaky tests per se, but identifies causes for a given flaky test. The tool executes a flaky test in different environment configurations. These configurations are composed of environment factors (e.g., memory sizes, CPU cores, browsers and screen resolutions) that are varied in each execution. The results are analysed using a spectrum-based localisation technique [45] that assigns a suspiciousness value and ranking to each configuration to determine the most likely factors that caused the flakiness. RootFinder [9] identifies potential causes of flakiness as well as the location in code that caused the flakiness. The tool can identify flakiness due to nine different causes, including network, time, IO, randomness, floating point operations and test order dependency. To do this, the tool instruments API calls during test execution, which can log interesting values (time, context, return value) and also add additional behaviour (e.g., add delays to trigger concurrency-related issues).

Several tools target concurrency as a source of flakiness. FlakeShovel [15] targets event races that can cause test flakiness by exploring different yet feasible event execution orders, however this is limited to GUI tests in Android apps. *Shaker* [13] exposes flakiness by adding noise to the environment in the form of tasks that also stress the CPU and memory whilst the test suite is executed. NodeRacer [14] is a tool for JavaScript programs running on `node.js` with which accelerates manifestation of event races that can cause test flakiness. It uses instrumentation and builds a model consisting of a happens-after relation for callbacks.

Another group of tools is designed to detect flakiness due to order-dependent tests. iDFlakies [11] uses reruns with randomised execution order, classifies all found flaky tests into either order-dependent or non-order dependent tests. DTDetector [6] presents four algorithms to identify test dependencies, which is manifested in test outcomes: reversal of test execution order, random test execution order, exhaustive bounded algorithm (which executes bounded subsequences of the test suite instead of trying out all permutations) and finally, the dependence-aware bounded algorithm that only tests subsequences that have data dependencies. iFixFlakies [10] aims to automatically fix order-dependent tests by utilising helper tests to reset the state required for order-dependent tests to pass. NonDex [7] targets implementation dependencies caused by assumptions developers make about under-determined APIs in the Java standard libraries, for instance the iteration order of hashed collections.

Herzig et al. propose using association rules to identify false test alarms [46] where test cases are composed of test steps and failing patterns of test steps leading to test failure imply a false test alarm. This is similar to our approach, with the difference that we directly enforce it with a runtime component.

While we do not directly target source of flakiness like concurrency, test order dependencies and under-determined APIs, those issues often come to the fore in certain environments, and therefore assumptions checking for such environments can be used to disable tests and therefore deal with flakiness as suggested in Section III.

B. Empirical Studies on Test Flakiness

There have been several empirical studies that investigated test flakiness and their root causes in open-source projects. The earlier study of Luo et al. [26] investigated the common causes and fixing strategies of flaky tests by mining test fixes from Apache projects' commit history. The study showed that concurrency, order-dependency, resource leaks and network dependency are responsible for most of the flaky tests observed.

A similar work by Gruber et al., [47] studied the presence of flaky tests in Python projects and found that order-dependency is, by far, the most common cause - responsible for 59% of the flaky tests in these projects. Other non-order-dependent tests are predominantly caused by network and randomness APIs. The study also noted that a typical rerun to detect flakiness is not an effective method. A recent study [48] investigated the major causes and fixes of flaky tests in JavaScript projects, noting that concurrency is the major know causes of flakiness in JavaScript. It was also found that flakiness due to order-dependency is not as prevalent in JavaScript as in Python [47]

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 12,2026 at 15:39:53 UTC from IEEE Xplore. Restrictions apply.

or Java [26]. Thorve et al., [49] identified five major causes of flakiness in Android application: concurrency, external dependencies, program logic, network, and UI, with concurrency bugs (36% of the flaky tests commits) being the major cause of flaky tests.

C. Flakiness caused by Network Dependencies

Network dependency (including connections, availability, and bandwidth) has been acknowledged as one of the key source of test flakiness [26], [27], [5]. This includes both local and remote network issues. Local issues pertain to managing resources such as sockets (e.g. contention with other programs for ports that are hard-coded in tests), while remote issues concern failures in connecting to remote resources (e.g., attempting to reach a web server). In a study of Python projects, Gruber et al. [47] noted that 413 tests are flaky due to network issues. It is also reported that 14% of the flaky tests found in six large-scale Microsoft projects are due to network related causes [50]. In a study consisting of Android projects [49], network is identified as a cause of flakiness of 8% of the detected flaky tests.

Flakiness caused by network instability or availability has been pointed at as a major issue for developers in two separate developers' surveys [27], [51]. Ahmad et al. [51] noted that one common strategy to reduce potential flakiness in tests that utilise network resources is by intentionally undermining network infrastructure for "worst case scenarios" (e.g., assumes that a connection to an external resource cannot be established or a port is not available). This way the test is prepared to handle potential flakiness caused by network issues by design.

D. Assumption Inference and Flakiness Control in the Wild

XUnit.SkippableFacts for xUnit.net [25] is similar to our network dependency extension for .NET written in C# in that it can skip tests if certain exceptions occur. It will not handle *failed* tests though.

Unruly [26] is a set of JUnit4 rules to deal with flaky tests. `QuarantineRule` simply retries failing tests until they pass or a maximum number of failure is reached. This only deals with some very specific causes of flakiness that occur in the same environment (randomness, concurrency). `ReliabilityRule` is the opposite – test only pass if the same test consistently passes a number of times.

The conditional test annotations in *JUnit5* in `org.junit.jupiter.api.condition` are closely related to our approach. The difference is that we aim to infer the conditions during speculative executions on-the-fly, whereas those assumptions can be checked before the test in being executed.

The closest existing technique we are aware of are the `@PendingFeature` and the `@NotYetImplemented` annotations in *spock* and *groovy* respectively, already briefly discussed in Section III-C. The semantics of those annotations is based on speculative executions where test failures are sanitised as they are seen as tolerable during development. Interestingly, for `@PendingFeature`, the opposite is also true – passes are converted into failures to indicate that the annotation has become redundant. This is closely aligned with the philosophy of test-driven development [52] where passing tests are used in the *definition-of-done*.

[25]https://github.com/AArnott/Xunit.SkippableFact

[26]https://github.com/unruly/junit-rules

VIII. Conclusion

We have presented a novel approach to address test flakiness by inferring assumptions which can sanitise tests that fail only due to environmental conditions that cannot be controlled in tests.

We have illustrated the utility of this approach and our proof-of-concept implementation *saflate* on six popular real-world Java and Groovy programs with tests relying on network availability. The results suggest that our method can achieve sanitisation with both high precision and recall, while the overhead to developers for using our method is considerably low, as it only requires adding a dependency and annotations to tests depending on networked resources. If *JUnit5* is used, even the use of annotations can be avoided and *saflate* can be deployed with only minor changes to the build scripts configuring the tests. Our results also suggest that the computational overhead imposed by our method is also low.

There are alternative use cases for our general approach such as existing sanitisation techniques for pending and not yet implemented features, therefore we believe that this method has some potential usage beyond dealing with network issues. A common theme between the use cases we have identified is instability that cannot be controlled and checked by engineers, in this respect, dealing with network dependencies and features that have not been completely implemented is similar.

We think that the technique we have proposed here can be useful in practice for projects that have *some* tests with network dependencies. If a large number of tests are being sanitised (e.g., in an application with heavy use of network dependencies), our method might not be appropriate for such a project. But for the programs we have investigated, the number of affected tests is relatively small, and often relate to marginal features. Then it can be argued that those tests failing should not block builds. At least, sanitised tests can be flagged and handled differently from failing tests in subsequent processes. For this reason, we believe that our method can add real value to industry practice.

Acknowledgment

This work was funded by a New Zealand SfTI National Science Challenge grant No. MAUX2004. The work of the first author was also supported by Oracle Labs, Australia.

References

[1] J. Micco. Flaky tests at google and how we mitigate them. [Online]. Available: https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html

[2] M. Harman and P. O'Hearn, "From start-ups to scale-ups: Opportunities and open problems for static and dynamic program analysis," in *2018 IEEE 18th International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2018, pp. 1–23.

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 12,2026 at 15:39:53 UTC from IEEE Xplore. Restrictions apply.

[3] J. Raine, "Reducing flaky builds by 18x," 2020, https://github.blog/2020-12-16-reducing-flaky-builds-by-18x/.

[4] R. Agarwal, L. Clapp, G. Korlam, M. K. Ramanathan, and V. Subramanian, "Handling flaky unit tests in java," 2021, https://eng.uber.com/handling-flaky-tests-java/.

[5] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "A survey of flaky tests," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 1, pp. 1–74, 2021.

[6] S. Zhang, D. Jalali, J. Wuttke, K. Muşlu, W. Lam, M. D. Ernst, and D. Notkin, "Empirically revisiting the test independence assumption," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, 2014, pp. 385–396.

[7] A. Gyori, B. Lambeth, A. Shi, O. Legunsen, and D. Marinov, "Nondex: A tool for detecting and debugging wrong assumptions on java api specifications," in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2016, pp. 993–997.

[8] A. Gambi, J. Bell, and A. Zeller, "Practical test dependency detection," in *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2018, pp. 1–11.

[9] W. Lam, P. Godefroid, S. Nath, A. Santhiar, and S. Thummalapenta, "Root causing flaky tests in a large-scale industrial setting," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2019.

[10] A. Shi, W. Lam, R. Oei, T. Xie, and D. Marinov, "ifixflakies: A framework for automatically fixing order-dependent flaky tests," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019, pp. 545–555.

[11] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "idflakies: A framework for detecting and partially classifying flaky tests," in *2019 12th ieee conference on software testing, validation and verification (icst)*. IEEE, 2019, pp. 312–322.

[12] J. Morán Barbón, C. Augusto Alonso, A. Bertolino, C. A. Riva Álvarez, P. J. Tuya González *et al.*, "Flakyloc: flakiness localization for reliable test suites in web applications," *Journal of Web Engineering*, 2020.

[13] D. Silva, L. Teixeira, and M. d'Amorim, "Shake it! detecting flaky tests caused by concurrency with shaker," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2020, pp. 301–311.

[14] A. T. Endo and A. Møller, "Noderacer: Event race detection for node. js applications," in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 2020, pp. 120–130.

[15] Z. Dong, A. Tiwari, X. L. Yu, and A. Roychoudhury, "Flaky test detection in android via event order exploration," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 367–378.

[16] M. Beller, G. Gousios, and A. Zaidman, "Oops, my tests broke the build: An explorative analysis of travis ci with github," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 2017, pp. 356–367.

[17] T. Durieux, C. Le Goues, M. Hilton, and R. Abreu, "Empirical study of restarted and flaky builds on travis ci," in *Proceedings of the 17th International Conference on Mining Software Repositories*, 2020, pp. 254–264.

[18] R. O. Rogers, "Scaling continuous integration," in *International conference on extreme programming and agile processes in software engineering*. Springer, 2004, pp. 68–76.

[19] T. Van Der Storm, "The sisyphus continuous integration system," in *11th European Conference on Software Maintenance and Reengineering (CSMR'07)*. IEEE, 2007, pp. 335–336.

[20] D. Ståhl and J. Bosch, "Modeling continuous integration practice differences in industry software development," *Journal of Systems and Software*, vol. 87, pp. 48–59, 2014.

[21] M. Fowler, "Eradicating non-determinism in tests," 2011, https://martinfowler.com/articles/nonDeterminism.html.

[22] E. Wendelin, "Identifying and analyzing flaky tests in maven and gradle builds," 2019, https://gradle.com/blog/flaky-tests/.

[23] "Gitlab testing guide – flaky tests," 2021, https://docs.gitlab.com/ee/development/testing_guide/flaky_tests.html.

[24] J. Bell, G. Kaiser, E. Melski, and M. Dattatreya, "Efficient dependency detection for safe java test acceleration," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 770–781.

[25] A. Gyori, A. Shi, F. Hariri, and D. Marinov, "Reliable testing: Detecting state-polluting tests to prevent test dependency," in *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, 2015, pp. 223–233.

[26] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2014, pp. 643–653.

[27] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, "Understanding flaky tests: The developer's perspective," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019, pp. 830–840.

[28] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE transactions on software engineering*, vol. 41, no. 5, pp. 507–525, 2014.

[29] E. Bruneton, R. Lenglet, and T. Coupaye, "Asm: a code manipulation tool to implement adaptable systems," *Adaptable and extensible component systems*, vol. 30, no. 19, 2002.

[30] C. A. R. Hoare, "An axiomatic basis for computer programming," *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.

[31] Y. Cheon and G. T. Leavens, "A simple and practical approach to unit testing: The jml and junit way," in *European Conference on Object-Oriented Programming*. Springer, 2002, pp. 231–255.

[32] C. Boyapati, S. Khurshid, and D. Marinov, "Korat: Automated testing based on java predicates," *ACM SIGSOFT Software Engineering Notes*, vol. 27, no. 4, pp. 123–133, 2002.

[33] C. B. Jones, "Specification and design of (parallel) programs," in *9th IFIP World Computer Congress (Information Processing 83)*. Newcastle University, 1983.

[34] J. C. Reynolds, "Separation logic: A logic for shared mutable data structures," in *Proceedings 17th Annual IEEE Symposium on Logic in Computer Science*. IEEE, 2002, pp. 55–74.

[35] K. Bojarczuk, N. Gucevska, S. Lucas, I. Dvortsova, M. Harman, E. Meijer, S. Sapora, J. George, M. Lomeli, and R. Rojas, "Measurement challenges for cyber cyber digital twins: Experiences from the deployment of facebook's ww simulation system," in *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2021, pp. 1–10.

[36] "Iso/iec 25010 quality model," https://iso25000.com/index.php/en/iso-25000-standards/iso-25010/, 2011.

[37] J. Bell and G. Kaiser, "Unit test virtualization with vmvm," in *Proceedings of the 36th International Conference on Software Engineering*, 2014, pp. 550–561.

[38] J. Mayer and R. Guderlei, "Test oracles using statistical methods," *Testing of component-based systems and software quality*, 2004.

[39] F. Hewson, J. Dietrich, and S. Marsland, "Performance regression testing on the java virtual machine using statistical test oracles," in *2015 24th Australasian Software Engineering Conference*. IEEE, 2015, pp. 18–27.

[40] S. Bechtold, S. Brannen, J. Link, M. Merdes, M. Philipp, J. de Rancourt, and C. Stein, "Junit 5 user guide- version 5.7.2," 2021, https://junit.org/junit5/docs/current/user-guide/.

[41] K. Kapelonis, *Java testing with Spock*. Simon and Schuster, 2016.

[42] J. Lampel, S. Just, S. Apel, and A. Zeller, "When life gives you oranges: Detecting and diagnosing intermittent job failures at mozilla," in *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)*. ACM, 2021, pp. 1381–1392.

[43] S. Versteeg, M. Du, J.-G. Schneider, J. Grundy, J. Han, and M. Goyal, "Opaque service virtualisation: a practical tool for emulating endpoint systems," in *2016 IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C)*. IEEE, 2016, pp. 202–211.

[44] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "Deflaker: Automatically detecting flaky tests," in *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*. IEEE, 2018, pp. 433–444.

[45] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Transactions on Software Engineering*, vol. 42, no. 8, pp. 707–740, 2016.

[46] K. Herzig and N. Nagappan, "Empirically detecting false test alarms using association rules," in *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, vol. 2, 2015, pp. 39–48.

[47] M. Gruber, S. Lukasczyk, F. Kroiß, and G. Fraser, "An empirical study of flaky tests in python," in *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2021, pp. 148–158.

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 12,2026 at 15:39:53 UTC from IEEE Xplore. Restrictions apply.

[48] N. Hashemi, A. Tahir, and S. Rasheed, "An empirical study of flaky tests in javascript," in *2022 38th IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2022.
[49] S. Thorve, C. Sreshtha, and N. Meng, "An empirical study of flaky tests in android apps," in *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2018, pp. 534–538.
[50] W. Lam, K. Muşlu, H. Sajnani, and S. Thummalapenta, "A study on the lifecycle of flaky tests," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 1471–1482.
[51] A. Ahmad, O. Leifler, and K. Sandahl, "Empirical analysis of practitioners' perceptions of test flakiness factors," *Software Testing, Verification and Reliability*, vol. 31, no. 8, p. e1791, 2021.
[52] K. Beck, *Test-driven development: by example*. Addison-Wesley Professional, 2003.

Authorized licensed use limited to: Istanbul Bilgi University. Downloaded on July 12,2026 at 15:39:53 UTC from IEEE Xplore. Restrictions apply.